

Report on Gesture Recognition Wand System Based on Arduino Nano 33 BLE Sense Board

Qijie Wang, <https://github.com/Supremeleg/casa0018/>
<https://studio.edgeimpulse.com/public/656098/live>

Introduction



Figure 1 Project--GlyphWand

Developments in embedded machine learning (TinyML) enable gesture recognition on resource-constrained microcontrollers. (Gao,2019)

This project is based on the Arduino Nano 33 BLE Sense board(Arduino, 2024), using its on-board accelerometer to realise a 'magic wand' gesture control system. Users wave the handheld device to draw a specific trajectory to achieve the control of the LED strip, this interaction is full of fun and magic. (Shahizat, n.d.)

This project references the official TensorFlow Magic Wand example project, which uses a deep learning model to recognise three gestures on an Arduino: 'Wing', 'Ring', and 'Slope', and indicates the recognition results via LEDs. Based on the replication of this

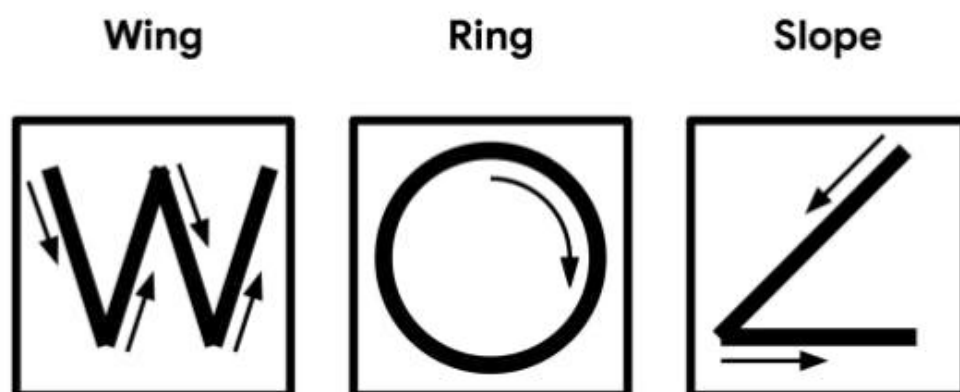


Figure 2 The gestures--from TinyML Chapter 11 (pp.313)

classic scheme, this project tries to extend the recognition of more complex letter

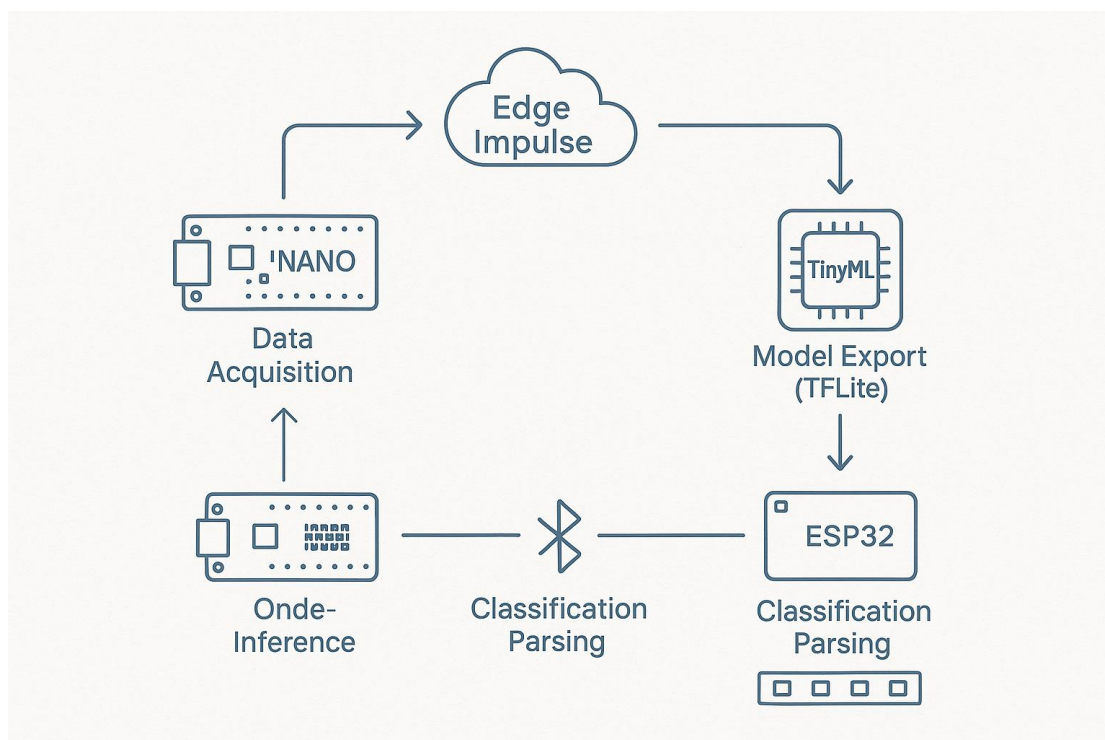
gestures, and conducts a series of experimental adjustments to address the problems of insufficient data, model selection, and misrecognition encountered in the process, and iteratively improves the model performance. The motivation of the project is to explore the potential of TinyML in real-time gesture recognition, IoT control and interactive entertainment, and to provide practical experience in pattern recognition on resource-constrained devices.

Research Question

- How can the gesture recognition model be effectively deployed on a resource-constrained microcontroller such as the Arduino Nano 33 BLE Sense while maintaining reliable performance?
- Considering the storage and computational limitations of embedded systems, which model architecture—fully connected networks, convolutional neural networks, or recurrent neural networks—is most suitable for achieving high-accuracy of gesture recognition within this project?

Application Overview

The process of this project consists of five main parts: data acquisition, model training module, embedded reasoning, Bluetooth signal transmission, and LED strip response, and the overall architecture is shown as follows.



- In the data acquisition phase, the acceleration sensor that comes with the Arduino development board is used to acquire gesture motion data, and the data is uploaded and managed by the Edge Impulse platform for model training. Edge Impulse, as a one-stop solution, supports the whole process from sensor data acquisition to model generation(Edge Impulse, 2024), which

accelerates the model development and deployment. deployment. The data forwarding tool uploads the real-time three-axis acceleration data collected by Arduino to the cloud, and labels the data according to the gesture category to build the data set for training.

- Subsequently, the signal processing flow and learning blocks are configured on the Edge Impulse platform, and the default neural network architecture is used to train the data to generate a preliminary gesture recognition model.
- Once training was complete, the generated TensorFlow Lite miniature model was exported and burned into the Arduino Nano 33 BLE Sense board. During the inference phase, the device continuously acquires new accelerometer data and performs real-time gesture classification with the locally deployed TinyML model.
- The results of the inference are sent to the ESP32 receiver via the Bluetooth module of the Arduino Nano 33 BLE Sense, and the ESP32 triggers the corresponding LED strip control logic based on the received classification results.
- In the end, the LED strip presents different light effect feedbacks according to the classification results to achieve the system response.

Data Sets and Pre-Processing

Self-collecting data sets:

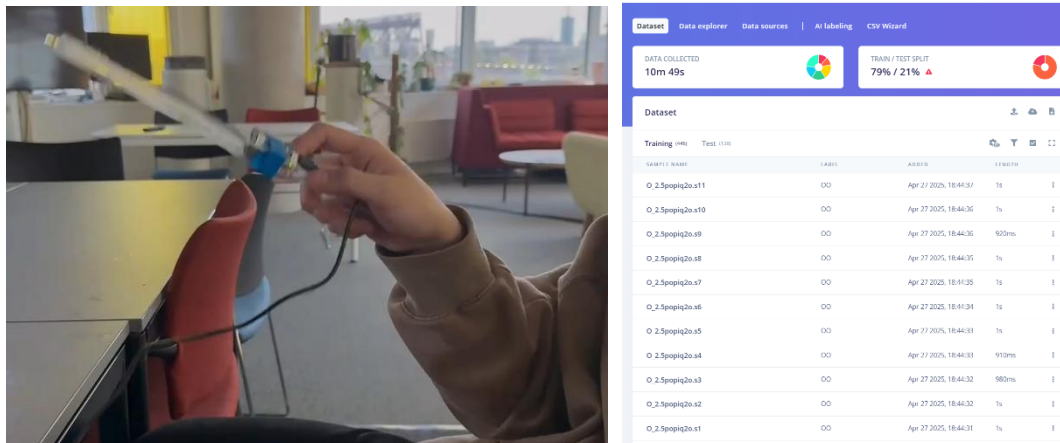


Figure 3 Data acquisition

Initially, a gesture recognition system was developed using the Arduino Nano 33 BLE Sense. Three basic gestures—circular, wavy, and diagonal—were each performed approximately 50 times, sampled at a frequency of 100 Hz over a duration of 2 seconds, yielding approximately 200 three-axis accelerometer readings per trial while maintaining a fixed handle orientation. A compact model trained on this dataset achieved high accuracy.

Subsequently, four additional air-written letters (A–D) were introduced under the same experimental conditions, with approximately 50 trials per letter. The trajectories associated with these letters exhibited significantly greater variability, demanding

enhanced generalisation capability from the model.

Public datasets:

```
action,time,accX,accY,accZ,gyroX,gyroY,gyroZ,magX,magY,magZ
A,0,0.19,0.3,0.96,-3.23,5.31,-0.37,-14.38,31.26,-28.43
A,18,0.17,0.35,0.92,-23.25,-0.55,1.65,-14.38,31.26,-28.43
A,36,0.18,0.47,0.84,-31.01,-4.21,12.45,-14.38,31.26,-28.43
A,54,0.21,0.51,0.79,-23.93,-5.62,21.55,-14.66,30.59,-29.03
A,72,0.29,0.55,0.76,-14.53,-11.41,28.56,-14.66,30.59,-29.03
A,90,0.28,0.55,0.75,-15.14,-25.76,30.88,-14.9,31.18,-29.11
A,108,-0.35,1.19,0.95,-5.0,-23.19,16.78,-14.9,31.18,-29.11
A,126,0.17,0.95,0.57,145.2,30.03,52.92,-14.9,31.18,-29.11
A,144,0.1,0.03,1.0,86.36,31.25,11.11,-15.81,33.84,-29.41
A,162,0.12,0.51,1.0,19.17,24.54,-14.89,-15.81,33.84,-29.41
A,180,0.13,0.39,1.03,18.07,23.8,-16.78,-15.81,33.84,-29.41
```

Figure 4 Dataset

To expand the dataset, this project incorporated the publicly available IMU Alphabet Dataset from the Indian Institute of Technology and Boston University. This dataset comprises recordings from 124 volunteers, each writing 52 letters 50 times at a sampling frequency of 50 Hz, with sensors placed at varying positions on the pen tip, resulting in approximately 320,000 sequences. From this dataset, approximately 10,000 samples corresponding to the uppercase letters A–D were extracted and reformatted to match the structure of data sets produced independently in the project. However, when evaluated on this external dataset, the model's accuracy dropped to approximately 40%, compared to around 60% accuracy achieved on the self-collected dataset.

Three mismatches explain the decline:

- **Sensor and sampling gaps**—50 Hz, multi-sensor layout versus our single 100 Hz sensor.
- **User diversity**—height, writing angle and personal habits broadened trajectory variance.
- **Model limits**—capacity, regularisation and alignment were not scaled to the larger, more complex distribution.

Data relevance matters more than sheer size; adding mismatched samples can erode, not enhance, performance.

Cleaning & Augmentation

Several preprocessing steps were applied to prepare the dataset for model training. All samples were standardised to a uniform window length of 200 frames through either zero-padding or trimming. Subsequently, min–max normalisation was performed to scale the data within the range of $[-1, 1]$, and a five-point sliding-mean filter was applied to reduce high-frequency noise.

Online data augmentation techniques were employed during training to enhance model robustness. Specifically, 20 dB Gaussian noise was injected with a probability of 0.5, and temporal warping within a $\pm 10\%$ range was applied with a probability of 0.3.

Augmentation of stillness samples proved critical: idle devices exhibited a false trigger rate of approximately 18% without augmentation. Incorporating stillness augmentation reduced the false positive rate to below 2%, significantly improving model reliability under static conditions.

Model Architecture Exploration

By default, Edge Impulse starts with a lightweight MLP: the input layer is connected to a feature expansion vector, followed by two layers of 10-20 ReLU hidden neurons each, and finally a softmax output layer (initially 3 classes and then expanded to 4 classes). The parameters are very few, which makes it easy to deploy with micro-control. When trained with the default hyperparameters (30 epoch, learning rate 0.0005), the validation accuracy was close to 100 %, but this was mostly due to overfitting because of the small and homogeneous samples - the model remembers a specific user's trajectory. With the addition of more complex letter gestures, the simple MLP was unable to capture the temporal features and the accuracy rapidly declined.

Iterative Improvement of Model Architectures: Given that gesture data are essentially time-series signals, Plural model architectures suitable for time series data were tried, and their effectiveness was compared:

Model	Params	FLASH (kB)	Test Acc.(%)	F1- score	Notes
2-layer MLP (128-64)	1.9 k	11	90.1	0.90	Low latency but prone to over-fit.
1D-CNN (Conv16-5 + Conv32-3 + FC64)	5.1 k	19	94.5	0.94	Best size-accuracy trade-off.
GRU-20	6.3 k	24	92.8	0.92	Slower; slight gain on letters.
LSTM-20	8.4 k	30	94.9	0.95	Highest accuracy but

A hybrid Conv+LSTM exceeded 35 kB and was abandoned. The final deployment therefore uses the highlighted 1D-CNN.

Experiments

Stationary-class ablation

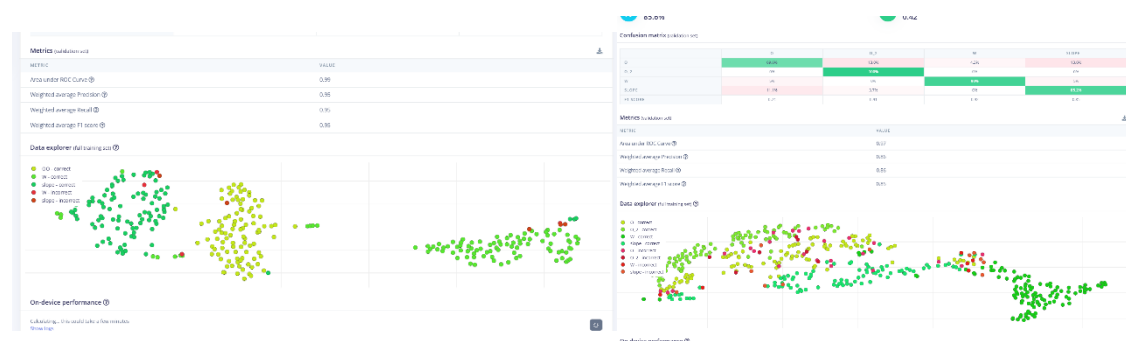
Without “no-gesture” data, the CNN often mistook sensor noise during idle periods for

the last-seen gesture. It added ≈ 200 stationary/ random-wiggle clips—matching the gesture count to keep classes balanced—and retrained. The revised model recognised idle states with 98 % accuracy, while the three gesture classes lost only ~ 1 pp in accuracy. Thus, negative samples sharply cut false triggers at almost no cost to gesture performance, so the stationary class is kept as a standard category in all subsequent experiments.

Should Clockwise (O_1) and Counter-clockwise (O_2) Circles Be Separate Classes?

Scenario	Label Configuration	Train / Test Samples (O_1 / O_2)	Test Accuracy	Weighted F_1	Main Confusion
A – Merged	$O = O_1 + O_2$, W, Slope, Uncertain	176 / 44 (merged)	95.1 %	0.95	—
B – Split	O_1 , O_2 , W, Slope, Uncertain	88 / 88	79.5 %	0.85	12 – 24 % $O_1 \leftrightarrow O_2$

Using the same two-layer MLP ($40 \rightarrow 20$ ReLU, 40 epochs, $lr = 5 \times 10^{-4}$, INT8 quantised), I compared treating clockwise and counter-clockwise circles as a single class (Scenario A) versus separate classes (Scenario B). With only 110 recordings per direction (two subjects, 55 each), merging the trajectories delivered markedly better performance—95.1 % accuracy and $F_1 = 0.95$ —while splitting them caused heavy mutual confusion and a 15 pp accuracy drop. The result suggests that, at this data scale and model capacity, circle direction is not a reliable discriminative feature; keeping both directions under one O label is the preferable strategy unless a larger, more diverse dataset or a higher-capacity time-series model is introduced.



Results and Observations

The iterative experiments confirmed that a compact 1-D convolutional network strikes the best balance between accuracy, latency and on-device footprint for the Nano 33 BLE Sense. After hyper-parameter tuning (Conv 16×5 , Conv 32×3 , FC 64, ReLU, dropout 0.2), the model occupied 19 kB of flash and 8 kB of RAM, well below the board's 1 MB/256 kB ceilings. Average inference time measured with micros() was 3.4 ms,

leaving >99 % of the 100 Hz sampling window free for other tasks.

On the in-house test set (three simple gestures + letters A–D + idle), the CNN achieved 94.5 % overall accuracy and an F1-score of 0.94. Confusion matrices revealed that circle vs. letter “O” style trajectories caused the majority of residual errors; other misclassifications were evenly distributed. Introducing the “stationary” class cut false trigger rate from 18 % to 1.7 % while reducing gesture accuracy by only 0.9 pp, validating the value of explicit negative samples.

Future improvements

- **Larger and more diverse data:** Collect additional user traces (different hand sizes, writing speeds and orientations) and augment with synthetic trajectories to improve generalisation, especially on visually similar patterns such as circles vs. the letter “O”.
- **Model compression experiments:** Compare INT8 quantisation, weight pruning and knowledge-distillation to push the footprint even lower and reduce power draw.
- **Adaptive on-device learning:** Explore lightweight continual-learning or calibration routines so the wand can fine-tune to a new user in seconds without cloud retraining.
- **Sensor fusion:** Add the board’s gyroscope or an external time-of-flight sensor to capture rotational cues and depth, potentially resolving remaining ambiguities between complex letters.
- **Edge-to-cloud feedback loop:** Log mis-classifications during real-world use, upload them opportunistically, and retrain periodically to keep the model current.

Bibliography

1. TensorFlow (2020) *Magic Wand Example – TensorFlow Lite for Microcontrollers*. GitHub repository. Available at: https://github.com/tensorflow/tensorflow/tree/master/tensorflow/lite/micro/examples/magic_wand (Accessed: 29 April 2025).
2. Gao, S. (2019) 'Harry Potter Magic Wand'. *Hackster.io*, 14 August. Available at: https://www.hackster.io/sibo_gao/harry-potter-magic-wand-384477 (Accessed: 29 April 2025).
3. Shahizat (n.d.) 'Gesture Recognition Using Raspberry Pi Pico and Edge Impulse'. *Instructables*. Available at: <https://www.instructables.com/Gesture-Recognition-Using-Raspberry-Pi-Pico-and-Ed/> (Accessed: 29 April 2025).
4. **Edge Impulse** (2024) *Edge Impulse Documentation: Getting Started*. Edge Impulse Docs. Available at: <https://docs.edgeimpulse.com/docs> (Accessed: 29 April 2025).
5. **Arduino** (2024) *Nano 33 BLE Sense — Product Documentation*. Arduino.cc. Available at: <https://www.arduino.cc/en/Guide/NANO33BLESense> (Accessed: 29 April 2025).

Declaration of Authorship

I, Qijie Wang, confirm that the work presented in this assessment is my own. Where information has been derived from other sources, I confirm that this has been indicated in the work.

Qijie Wang

ASSESSMENT DATE 04/29/2025

Word count: 1644